INSIGHTFLOW

CS 331 – Software Engineering Lab

Assignment 4 – Software Architecture Design (Q1)

Q1. Chosen Architecture Style and Justification

Chosen Architecture Style: **Microservices Architecture**. InsightFlow is an AI-powered system for PDF ingestion, RAG-based chat-with-PDF, and report generation. Microservices is the best fit because the system contains multiple independent responsibilities (ingestion, embedding, vector search, RAG, reporting, and audit), each of which benefits from independent deployment, scaling, and lifecycle management.

1. Granularity of Components

The system is split into small services, each owning a single responsibility (high cohesion) and a well-defined API (loose coupling). This helps in development, testing, and deployment. Main components are listed below.

- Ingest Service – PDF upload, text extraction, chunking (owns original files metadata).
- Embedding Service – Calls OpenAI for embeddings; batches and retries; provides embeddings API.
- Vector Store (Qdrant) – Stores vectors and payload; supports upsert, search, delete-by-filter.
- RAG / Query Service – Orchestrates retrieval, builds prompt context, calls LLM for answers.
- Report Generation Service – Cleans processed numerical data and produces downloadable reports (PDF/CSV).
- DataProcessor – Validation and cleaning logic for numeric inputs (separate to keep processing fast).
- History / Audit Service – Append-only logs of uploads, queries, report generation for traceability.

2. Scalability, Maintainability, Performance

Scalability: Services can be scaled independently (e.g. RAG scaled on queries; Embedding scaled on ingest).
Maintainability: Smaller codebases per service reduce cognitive load and speed up fixes and feature work.
Performance: Heavy tasks (embeddings, vector search, LLM calls) are isolated and can be optimized individually.

3. Fault Isolation, Consistency & Resilience

Microservices provide fault isolation: a failure in one service (e.g., Embedding) does not take down others. For cross-service data consistency, we use eventual consistency patterns where appropriate (e.g., vector upserts after ingest). Important resilience patterns include retries with exponential backoff for external calls (OpenAI), circuit breakers, and bulkheads.

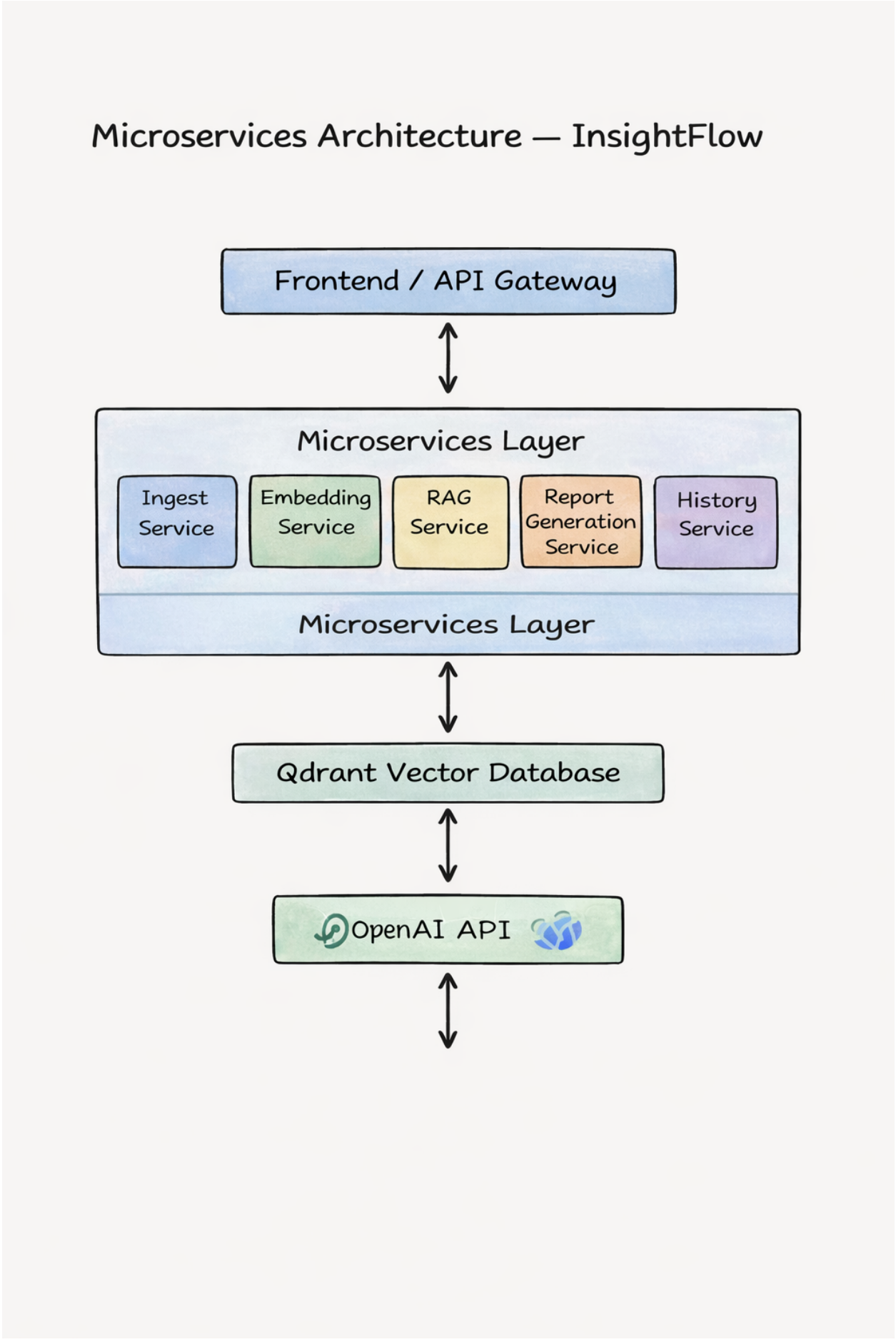
4. Security, Observability and DevOps

Security: Use environment-based secrets (no hard-coded keys), network policies, and restrict access to Qdrant.
Observability: Application logs, distributed tracing (e.g., OpenTelemetry), and metrics (Prometheus/Grafana).
DevOps: Each service has CI/CD pipelines, container images, and versioned APIs.

5. Comparison with Monolithic Architecture (short)

A monolith would be simpler initially but would cause deployment and scaling pain later. Given InsightFlow's need for independent scaling of embeddings/queries and frequent updates to AI prompts/models, microservices provide long-term benefits.

Architecture Diagram (Improved)



Checklist for Full Marks

- Architecture style clearly named and justified.
- Granularity of components listed and explained.
- Scalability, maintainability and performance discussion included.
- Fault isolation, consistency and resilience patterns mentioned.
- Security, observability and DevOps considerations added.
- Short comparison with alternative (monolith) included.
- A clear architecture diagram is provided.